How a Robot Finds Its Way

Pedro Pathing 3 for FIRST Tech Challenge

Deck A: a live demo first, then the big ideas behind it.

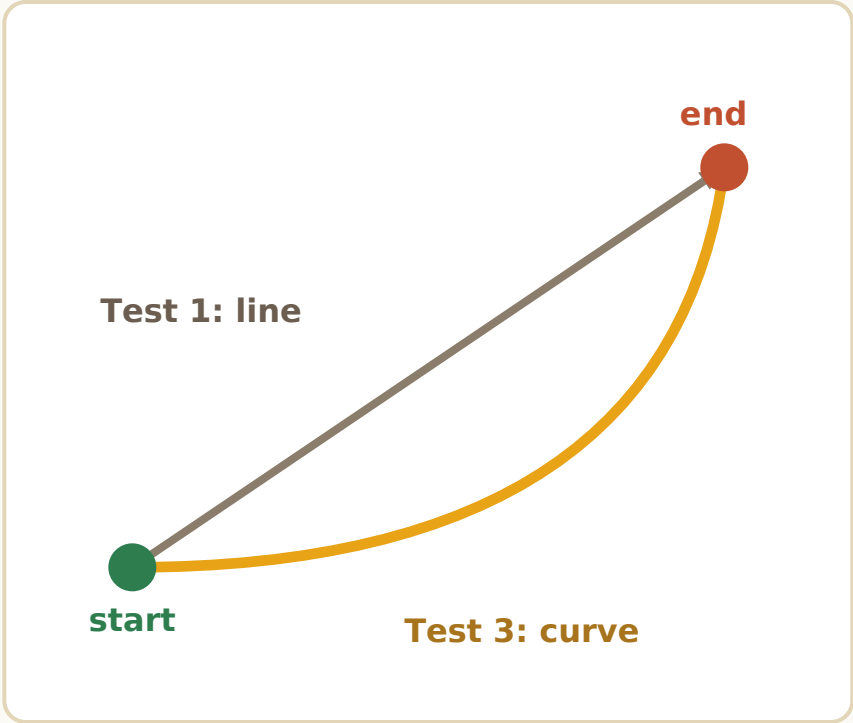


Built from pedropathing.com/docs, the goBILDA Pinpoint user guide and our PathLab demo.
Deck B shows how to use it in code.

THE DEMO

Today's demo: a robot drives a path.

1. A robot drives from a **start spot** to an **end spot** on the real field.
2. First a **straight line** (Test 1). Then a **curve** (Test 3).
3. At the end it waits 2 seconds, then drives back to the start.



Read more: Our PathLab project: Path Tests User Guide (github.com/smrc-pune/PathLab)

THE DEMO

While you watch, write in your notebook.

I saw…

• Where did the robot start? Where did it stop?

• Which way was the front pointing at the start, on the way, and at the end?

• What was different between the line and the curve?

I wonder…

• How did it know where to go?

• How did it know where it was?

• Write your own question.

My guess…

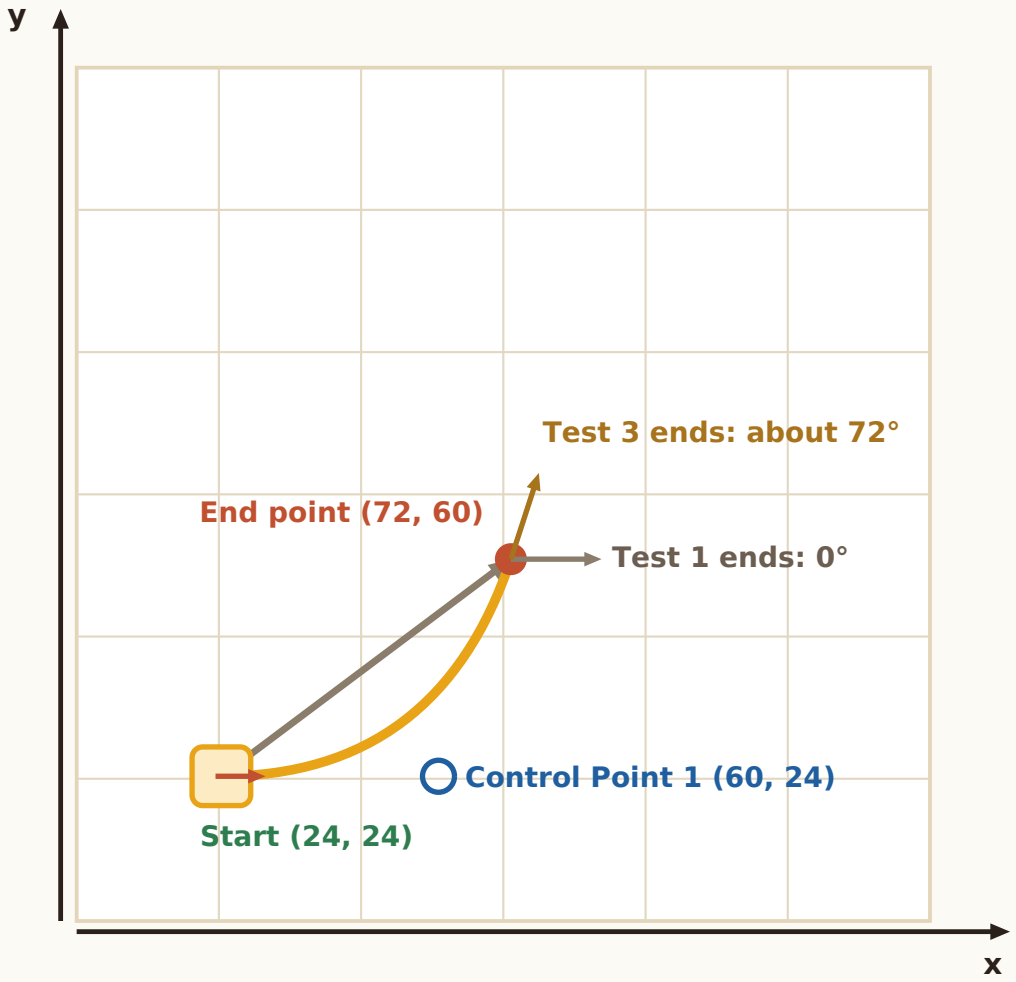
• How might the robot work this out? Draw it or write it.

• What could go wrong?

Read more: Instructor notes have a suggested discussion plan.

THE DEMO

Here is what the robot did.



Start	(24, 24), facing 0° (right)
End point	(72, 60)
Test 1: line	Straight, about 60 in. Keeps facing 0° the whole way.
Test 3: curve	Bends towards Control Point 1 (60, 24). About 65 in. Ends facing about 72°.
Then	Waits 2 seconds, then drives back to the start.

Check your notes. What did you see that is not on this list?

Read more: PathLab Path Tests User Guide: default points. If points were edited on the Dashboard, yours will differ.

WHY — THE PROBLEM

Why "drive for 2 seconds" fails

What changes	What it does to a timed move
The battery gets lower	The motors turn slower, so the same 2 seconds covers less distance.
A wheel slips	The wheel turned, but the robot did not move as far as it should have.
Another robot bumps you	You are now somewhere new, and the code has no idea.
The floor is different	Grip changes, so the same power moves the robot differently.

> Timing counts how long the motors ran. It does **not** measure where the robot is.

BACK TO THE DEMO The robot in the demo drove out, waited 2 seconds and drove back. Could “drive for N seconds” do that, run after run, to the same spots?

Read more: This slide is general robotics reasoning, not a claim from the Pedro docs.

WHY — THE GOAL

Arrive at the same spot, every match.

1

Know where I am

Measure the robot's position all the time, not just guess it.

2

Know the route

Describe the route as a path on the field.

3

Keep correcting

If the robot gets pushed off, steer back onto the path.

Pedro Pathing is a path-following library for FTC. Its docs say it follows paths while correcting for disturbances, and that it plans its braking using measurements of the robot.

BACK TO THE DEMO

Did the robot come back to the same spot it started from? How could we check? (Tape an X at the start and measure.)

Read more: Pedro docs → Introduction

THE MAP IDEA

Your phone's map app already does this.

Map app	Pedro Pathing	In our code
Your address	A **Pose**: a spot plus which way you face	`Pose`
Map settings (miles or km)	A **PoseFactory**: degrees or radians, normal or mirrored	`PoseFactory`
The blue dot	The **localizer**: where am I right now?	`PinpointLocalizer` `follower.pose()`
The route	A **path**: a line or curve between two Poses	`line()` `curve()`
The voice and the driving	The **follower**: steers the robot along the path	`Follower`
"Recalculating…"	Correcting back onto the path after a push	`Follower` (Foresight)

BACK TO THE DEMO

On the Dashboard picture, find the planned path (the route) and the robot’s position. Which is the plan and which is the reality?

Read more: Pedro docs → Setting Up Your Autonomous, Creating a Pose, Creating a Path

THE BIG PICTURE

Pedro Pathing is a box.

INPUTS

Sensor readings
x, y and heading from the Pinpoint, again and again.

Starting Pose
Where you tell Pedro the robot begins.

Your path
The lines and curves you drew, built from Poses.

INSIDE THE BOX

PEDRO PATHING

Localizer
Where am I?

Follower
How do I get there?
brakes early using Foresight

runs again on every `loop()`

OUTPUT

Wheel powers
How hard each drive motor pushes.



The robot moves. The Pinpoint measures again.

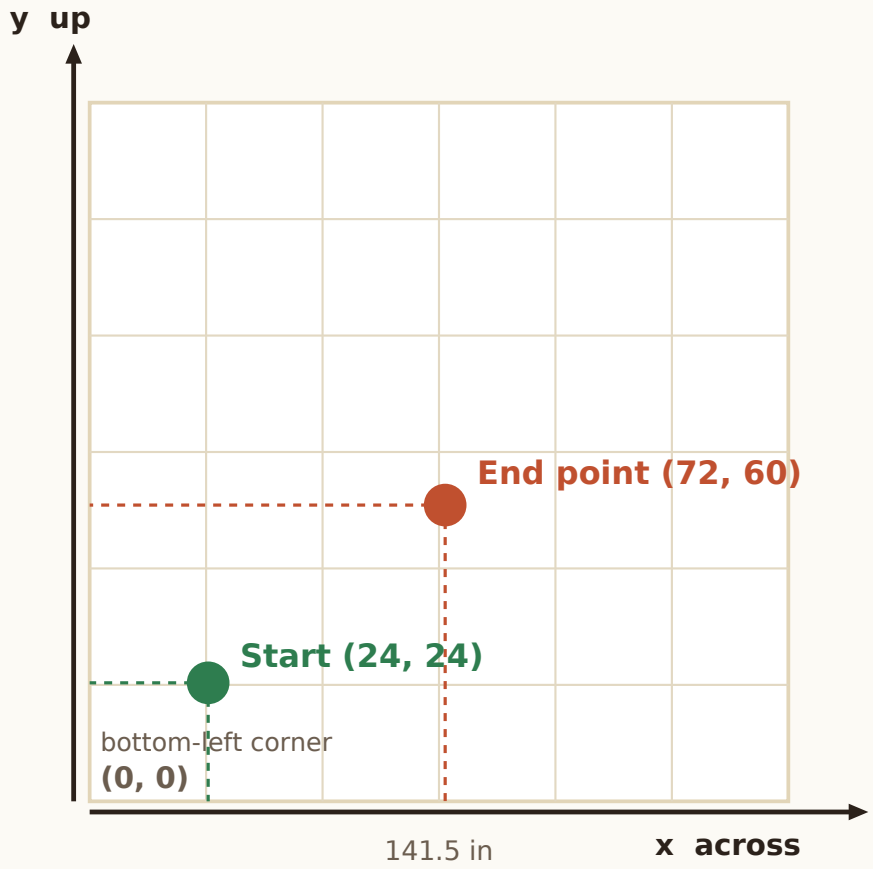
Localizer = the robot's GPS. Follower = its driver. A driver is only as good as the GPS.

BACK TO THE DEMO

The path you saw drawn on the Dashboard was an **input**. The wheels turning was the **output**. Everything in between happened inside the box.

Read more: Pedro docs → Introduction. The box picture is our own simplification, not a diagram from the docs.

The field is a grid.



x	how far across the field, in inches
y	how far up the field, in inches
(0, 0)	the bottom-left corner
Field	141.5 in × 141.5 in

Like a game of Battleship: **two numbers find one square.**

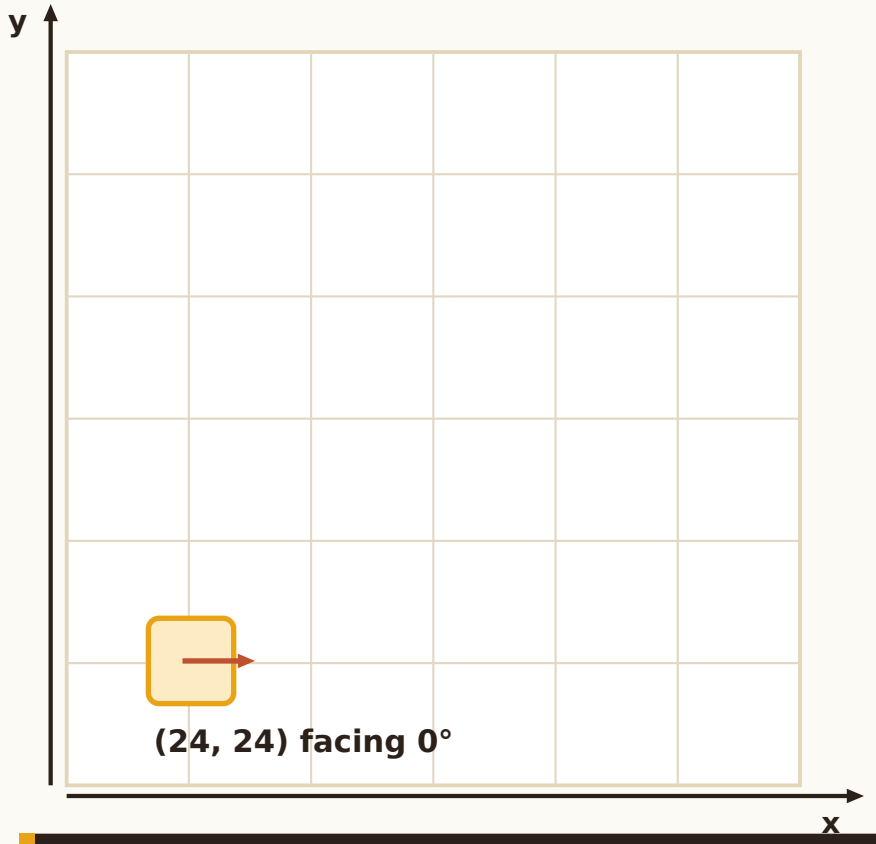
Pedro uses its **own** coordinates, not the official FTC ones. Examples from other tools may use different ones, so check before copying numbers.

BACK TO THE DEMO

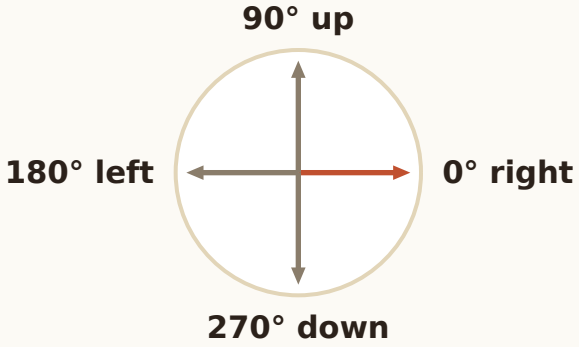
The demo started at (24, 24) and went to (72, 60). Find both on the grid. About how many tiles is that across and up? (One tile is about 23.6 in.)

Read more: Pedro docs → Coordinates · Pedro Visualizer (field is 141.5 in × 141.5 in)

A Pose is a spot plus a facing.



x	how far across (inches)
y	how far up (inches)
heading	which way the front points (degrees)



Turning **counter-clockwise** makes the heading go **up**.

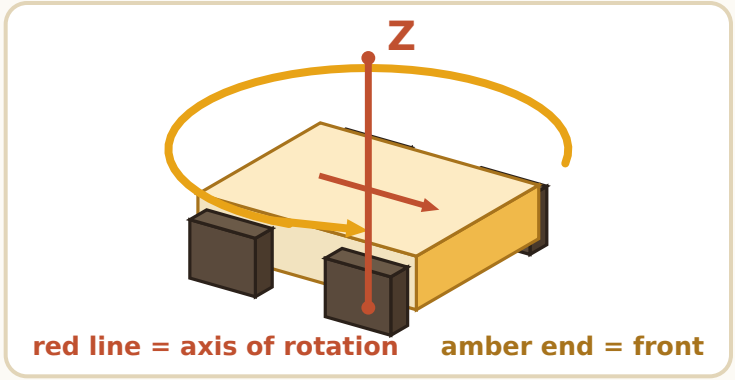
Same spot, different Pose. Stand still and turn: x and y stay the same, heading changes.

BACK TO THE DEMO

The demo started at (24, 24) facing 0°. The End point on the Dashboard has heading 90°, yet in Test 1 the robot stayed facing 0°. Why? Hold that thought.

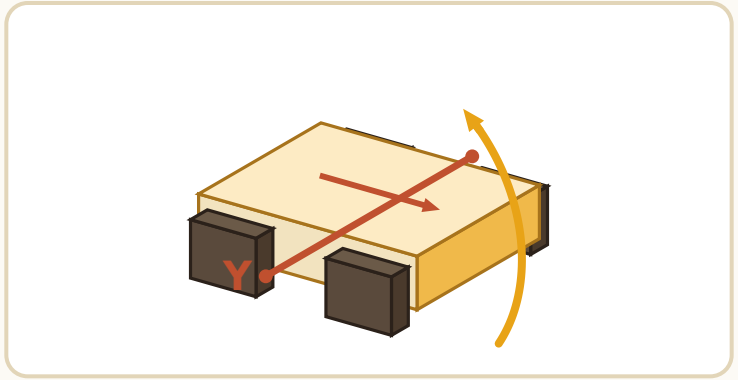
Read more: Pedro docs → Creating a Pose, Coordinates

Three ways to turn. One is the heading.



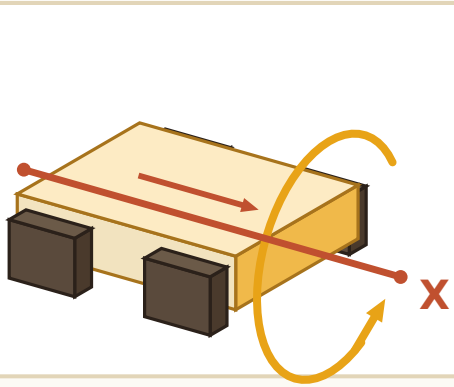
Yaw

Shake your head “no”.
Z axis, straight up and down.



Pitch

Nod “yes”.
Y axis, side to side.



Roll

Tilt your head.
X axis, front to back.

What is an IMU?

Inertial Measurement Unit. A tiny chip that feels motion. The Pinpoint has one built in.

How does it work?

A **gyroscope** inside feels how fast the chip is turning. The computer adds up the turning, bit by bit, to get the heading.

Why do we need it?

On a flat field only **yaw** matters, and yaw is the robot’s **heading**. At power-up the IMU calibrates, so keep the robot still.

BACK TO THE DEMO — The heading numbers in the demo (0°, 90°) are yaw, measured in degrees.

Read more: Our interactive: “Yaw, Pitch & Roll” (claude.ai/artifact/WejHr38cdhPF55tcS3JExx) · goBILDA Pinpoint guide

A PoseFactory makes every Pose the same way.

Think of a **stamp**. Set the rules once, and every Pose you stamp follows them.

```
PoseFactory p = PoseFactory.degrees();

Pose startPose = p.of(24, 24, 0);
Pose endPose = p.of(72, 60, 90);
```

The order is always **x, y, heading**.

> Declare your Poses once, at the top of the class. Not inside `loop()`.

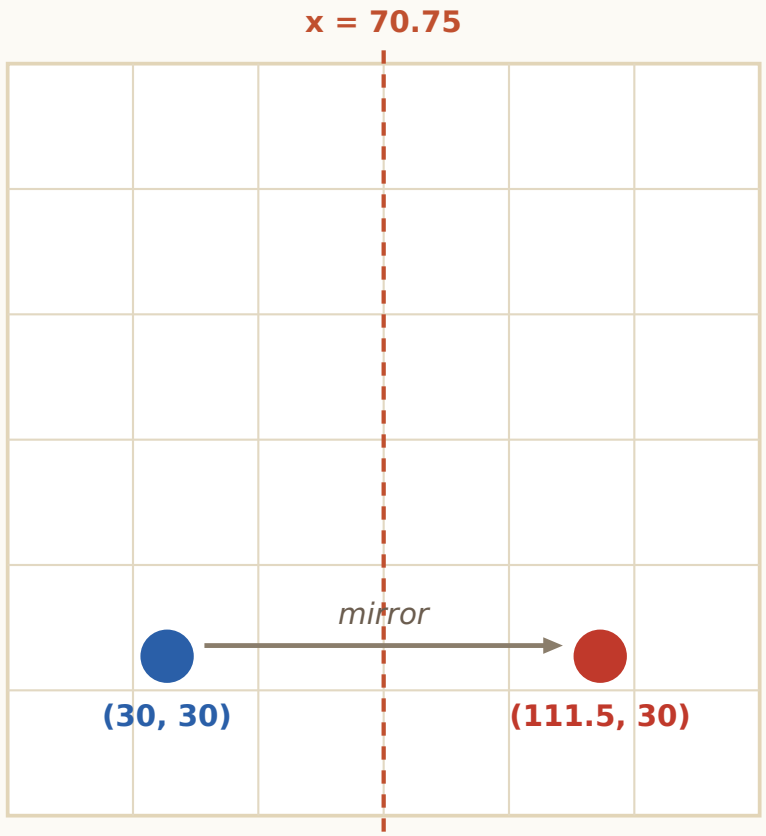
A factory settles…	Example
The unit for heading	`degrees()` or `radians()`
Whether poses are flipped	next slide: `mirrorX()`

> **Watch the units.** You *type* degrees into the factory, but `follower.pose()` hands heading back in **radians**.

BACK TO THE DEMO On the Dashboard you type the points in degrees. The demo code uses PoseFactory.degrees() for every pose, so nobody has to type radians.

Read more: Pedro docs → Creating a Pose, Pose Factory

Draw it for one alliance. Flip it for the other.




```
PoseFactory p = PoseFactory.degrees();
p = p.mirrorX(70.75);
```

`mirrorX(line)` flips every Pose across the line **x = line**. **70.75** is the middle of the 141.5-inch field.

> The docs describe positions flipping. They do **not** say what happens to headings, so always check a mirrored path in the Visualizer before you run it.

BACK TO THE DEMO

Try it on the demo's start: mirrored across x = 70.75, where is (24, 24)? [(117.5, 24)] What happens to its heading? We would check in the Visualizer.

Read more: Pedro docs → Pose Factory

Odometry: add up every little movement.

1. **A small wheel rolls on the floor.** It has no motor. It just rolls as the robot moves.

2. **An encoder counts how far it turned.** Distance = turns × length around the wheel.
   Example: 5 turns of a wheel that is 100 mm around = 500 mm.

3. **The computer adds up all the tiny moves,** starting from the Pose you gave it. The total is where the robot is now.

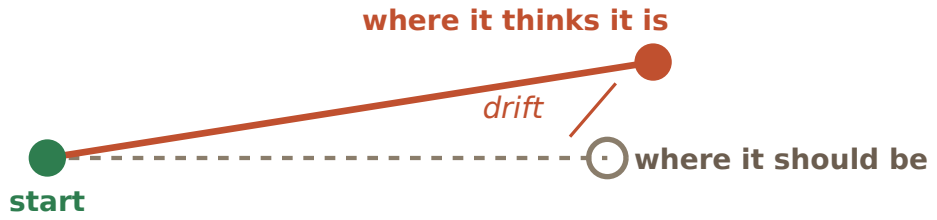
A **forward pod** (x) + a **sideways pod** (y) + the **IMU** (heading) = a Pose.



A goBILDA odometry pod

Example: goBILDA's 4-Bar pod has a **32 mm** wheel (about 100 mm around) and counts **2,000** per turn. One count is about **0.05 mm**.

Photo: goBILDA



Each reading has a tiny error, and the errors add up. This is **drift**.

BACK TO THE DEMO

After the run, did the robot land exactly on the end point? The telemetry reported how far off the path it was. Off the path according to whom?

Read more: goBILDA Pinpoint User Guide and 4-Bar Odometry Pod page (gobilda.com) · the rest is general physics

Meet the goBILDA Pinpoint.



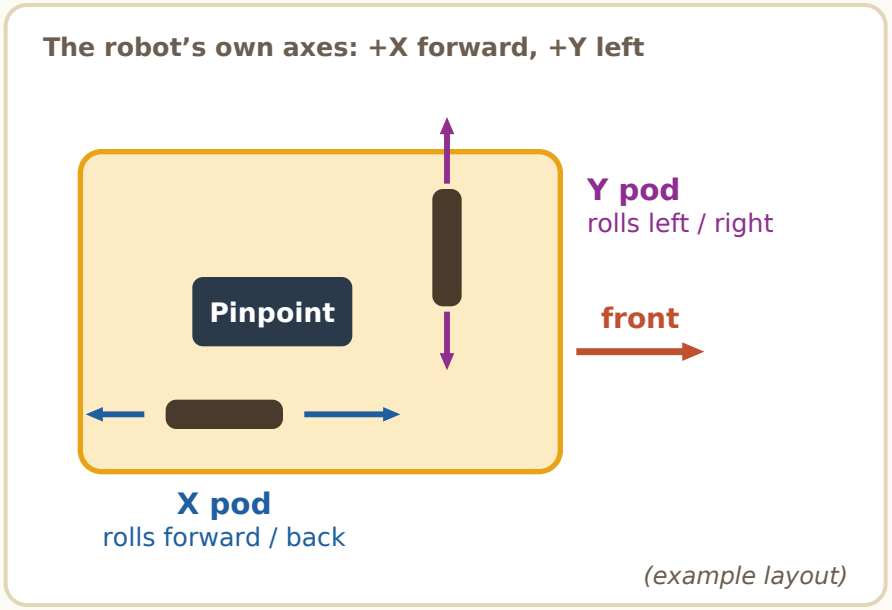
Part	What it does
Odometry pods (two)	Small wheels with no motor. Each one counts how far it rolls.
Pinpoint computer	Reads both pods and its built-in IMU. Combines them **1,500 times a second** and reports x, y and heading.
Ports on the Pinpoint	**X** and **Y** for the two pods, and **I²C** to the robot's controller. They are printed on the board.

> goBILDA expects the robot to land within about **10 mm (0.5 in)** of where it started after being driven around and brought back.

BACK TO THE DEMO Every position number in the demo came from the Pinpoint. Point to where it sits on our robot.

Read more: goBILDA Pinpoint User Guide · photo: goBILDA (gobilda.com) · Pedro docs → Pinpoint

Which pod measures what.



Pod	What it measures	Plugs into
X pod (forward)	Forward and back movement	X port
Y pod (sideways)	Left and right (strafe) movement	Y port

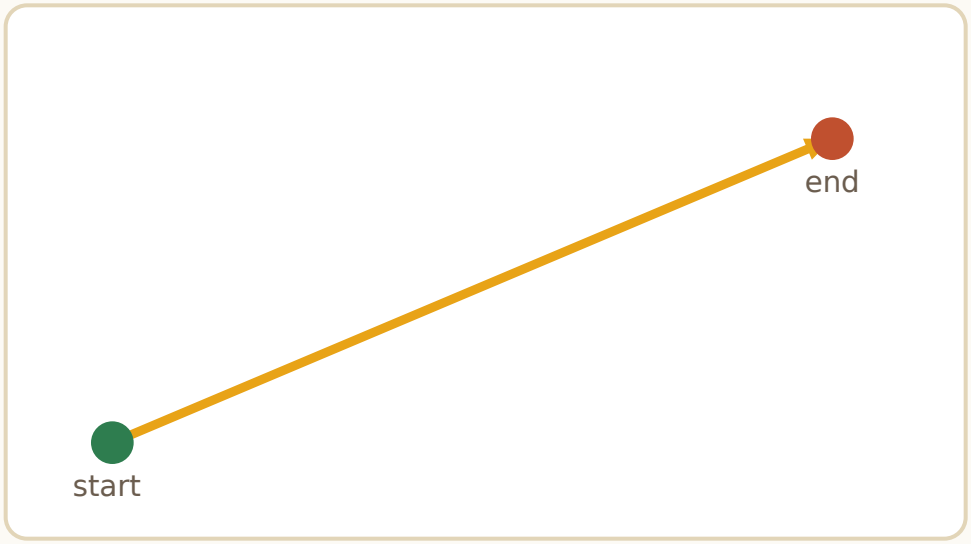
The names say what each pod **measures**, not where it sits on the robot.

Where the pods sit on the robot is measured once, during tuning. See our **Tuning Checklist**.

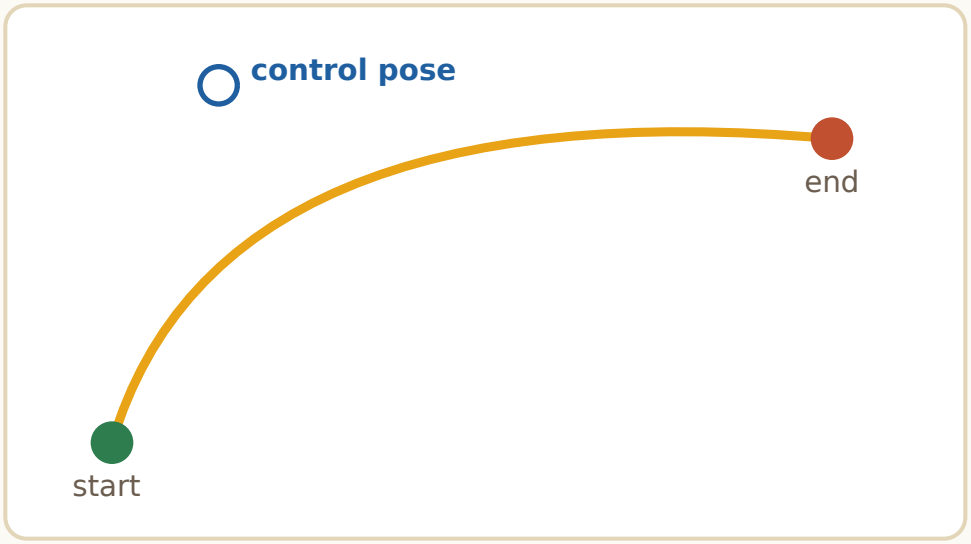
Read more: goBILDA Pinpoint Odometry Computer User Guide (3110-0002-0001), setup and offsets

A path is a line or a curve between two Poses.

line()



curve()



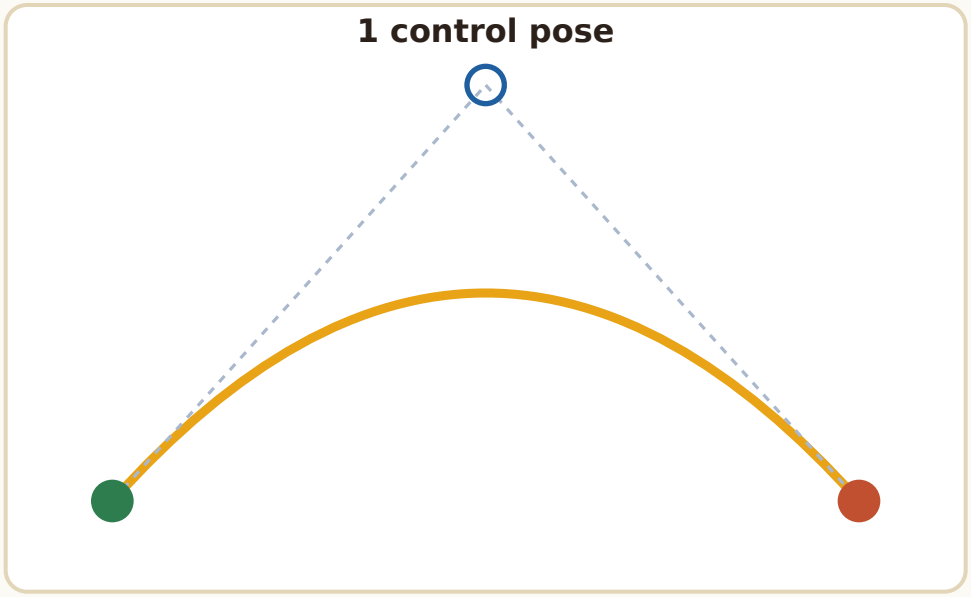

```
line(startPose, park)
curve(startPose, controlPose, park)
```

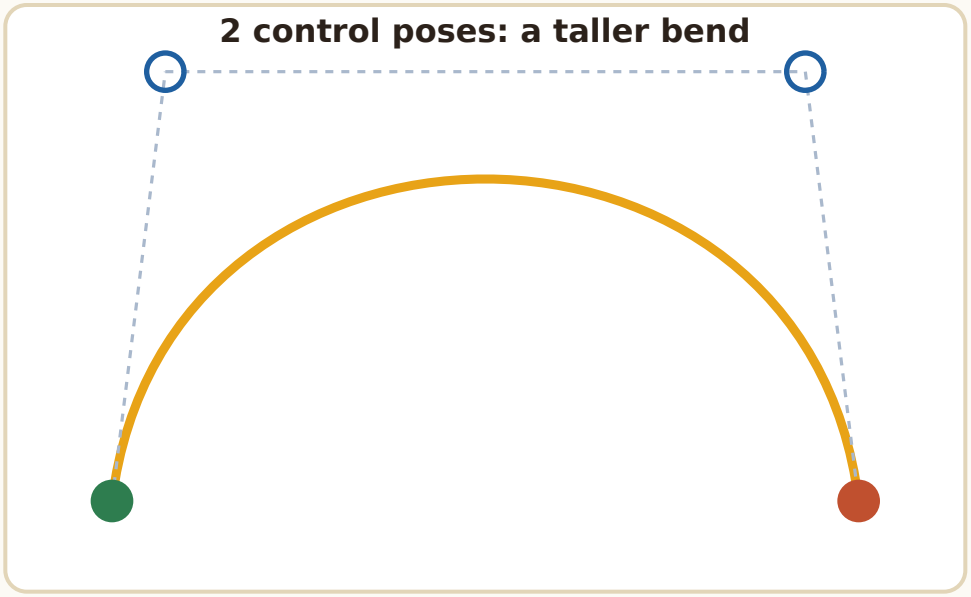
BACK TO THE DEMO

Test 1 used Paths.line(start, end). Test 3 used Paths.curve(start, controlPoint1, end). Same start, same end, different path.

Read more: Pedro docs → Creating a Path · Visualizer: visualizer.pedropathing.com

A control pose pulls the path towards it.





Think of a rubber band. Add a control pose in the middle and the straight line bends towards it. Add more to bend it further.

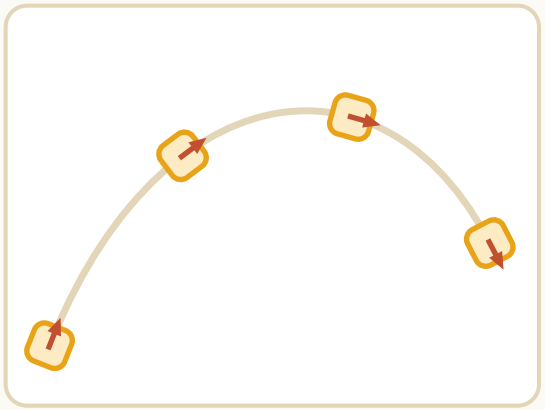
BACK TO THE DEMO

In Test 3 the path bent towards Control Point 1 at (60, 24). Predict: if we move it, what changes? Then edit it on the Dashboard and run Test 3 again.

Read more: Pedro docs → Bézier Curves · try it: visualizer.pedropathing.com

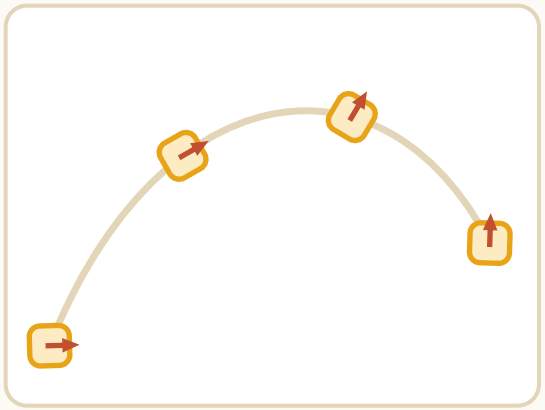
Where you face is separate from where you go.

A car always faces where it drives. A mecanum robot can slide sideways while facing another way, so **you choose** the heading along the path.



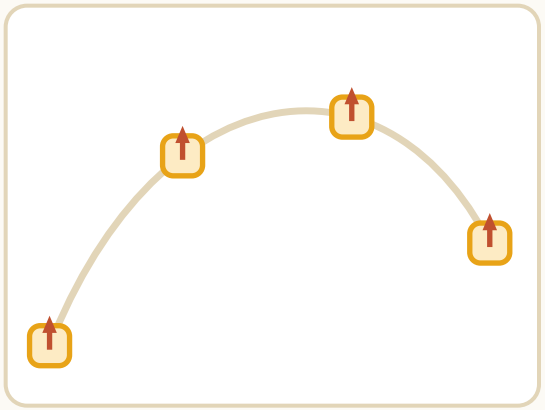
`tangent`

Faces the way it is travelling. The default.



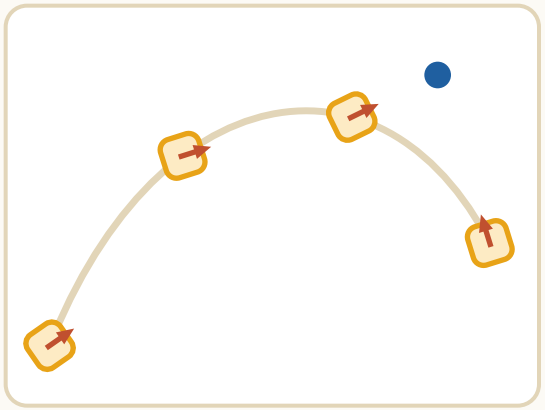
`linear`

Turns steadily from the start heading to the end heading.



`constant`

Keeps one heading the whole way.



`facingPoint`

Always faces one chosen point.

Go deeper: `piecewise` mixes these: for example tangent for the first half, then linear.

BACK TO THE DEMO

You saw constant (Test 1) and tangent (Test 3). PathLab also has Test 2 (linear) and Test 7 (facing a point). Try those next.

Read more: Pedro docs → Interpolation, Creating a Path

The follower drives. It updates every loop.

```
private Follower follower;

@Override
public void init() {
    follower = Constants.create(hardwareMap);
    follower.setPose(startPose);
}

@Override
public void loop() {
    follower.update();
}
```

`follower.update()` **must run on every loop.** Skip it and Pedro stops knowing where the robot is.

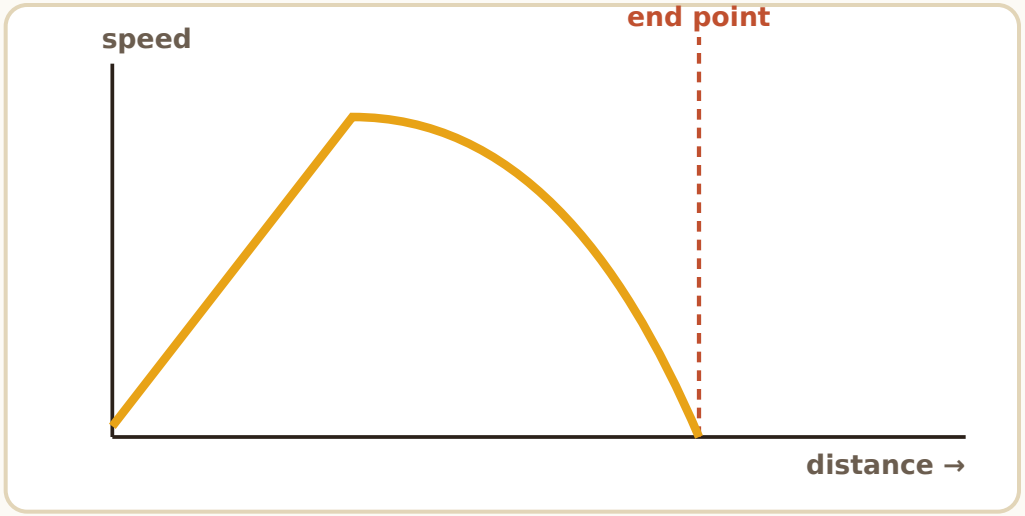
`setPose` tells Pedro where the robot *starts*. It must match where you really put it.

Do not freeze `loop()` with waits or `while` loops. The follower would stop getting updates.

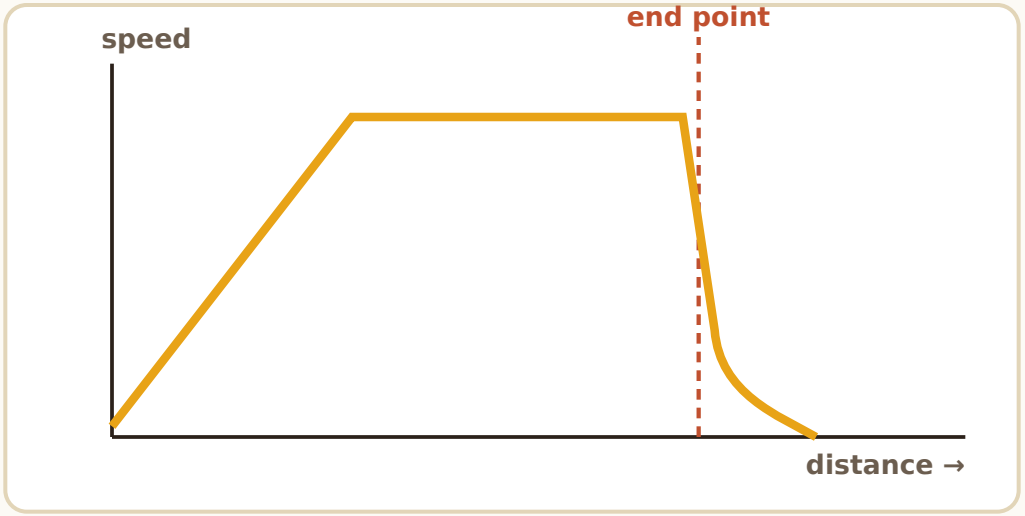
BACK TO THE DEMO In the demo, pressing A starts the path with follower.follow(). The OpMode keeps calling follower.update() and waits while follower.isBusy().

Read more: Pedro docs → Setting Up Your Autonomous, TeleOp Usage

Foresight: a good driver brakes early.



Slow down early: stops on the end point.



Slow down late: slides past it.

Foresight is Pedro 3's path-following algorithm. During tuning the robot is driven and braked to measure how far *this* robot slides. Pedro uses that to start slowing at the right moment. See our **Tuning Checklist**.

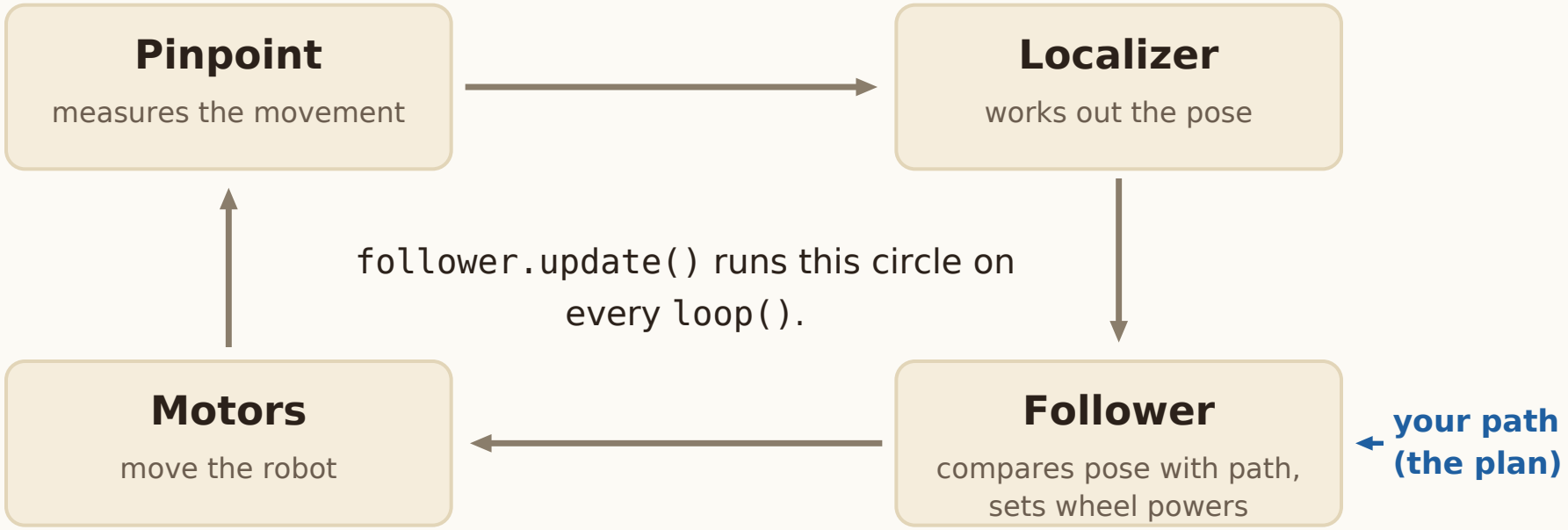
BACK TO THE DEMO

Watch the end of the run again. Does the robot slow down before the end point? Does it stop on it, or slide past?

Read more: Pedro docs → Introduction, Foresight · Pedro paper (Sripada & Henderson), footnote 2

PUTTING IT TOGETHER

One loop. Every pass.



Delete one box and the circle breaks. No Pinpoint means no position. No follower means no steering.

BACK TO THE DEMO

The Dashboard shows the plan (the path) and the robot's position. The gap between them is what the follower keeps working to close.

Read more: Pedro docs → Introduction, Setting Up Your Autonomous

BACK TO THE DEMO

Now answer your own questions.

A question you may have asked	Where the answer is
How did it know where to go?	It was given Poses and a path. *(slides 10, 17–18)*
How did it know where it was?	The Pinpoint counts wheel turns and senses heading; the localizer turns that into a pose. *(14–15)*
Why did it curve?	A control pose pulled the path towards it. *(18)*
Why did it face that way?	The heading mode: constant in Test 1, tangent in Test 3. *(19)*
Why was it not exactly on the line?	Small measuring errors, and the follower correcting them. *(14, 20)*
How did it stop at the end point?	The follower steers and brakes using its pose. Foresight plans the braking. *(21)*
Why did it come back by itself?	Our OpMode is a state machine: ready, driving, pausing, returning. That is code, so it is in Deck B.

Run the demo once more. What do you notice now that you missed the first time? Which of your questions are still open?

Read more: Slide numbers refer to this deck. The state names come from the PathLab design notes.

RECAP

Fifteen words you now own.

Pose	x, y and heading: a spot plus a facing
heading	which way the front of the robot points
yaw	turning in place: this is the heading
IMU	a chip that feels how fast the robot is turning
localizer	answers "where am I?"
Pinpoint	the goBILDA sensor that measures movement
odometry	measuring movement from small rolling wheels
drift	small errors that add up over time

path	a line or curve between two Poses
control pose	the point that bends a curve
heading interpolation	how the heading changes along the path
follower	answers "how do I get there?"
Foresight	Pedro 3's path-following algorithm
PoseFactory	makes Poses with the same units and mirroring
`loop()`	where the follower must update every pass

CHECK YOURSELF

Six questions. Say the answers out loud.

1	Why does "drive 2 seconds at 50% power" give a different result from match to match?
2	A robot stands still and turns left 90°. Which parts of its Pose change?
3	You built a blue-side auto. What does a mirrored PoseFactory save you?
4	Odometry is like counting steps with your eyes closed. What goes wrong on a long walk?
5	Why must the follower update on every single loop?
6	In the demo, Test 1 and Test 3 ended at the same spot. Test 1 ended facing 0° and Test 3 about 72°. What was different about the paths?

Read more: Answers are in the instructor notes.